SETUP GUIDE

Atlassian MCP Server

Install Docker, run the server, and connect Claude Desktop, Claude Code, and ChatGPT — from scratch.

116 tools across Jira · Bitbucket · Confluence · Bamboo

Atlassian MCP Server — Setup Guide

Atlassian MCP Server — Full Setup Guide (From Scratch)

A complete, beginner-friendly walkthrough: install everything from zero, run the server (locally or in Docker), and connect it to **Claude Desktop**, **Claude Code**, and **ChatGPT**.

Table of Contents

1. What you'll build
2. Prerequisites — install everything
3. Get the project
4. Configure credentials (.env)
5. Choose your run mode
6. Run with Docker (recommended)
7. Run locally with Python (no Docker)
8. Connect to Claude Code (CLI)
9. Connect to Claude Desktop
10. Connect to ChatGPT
11. Verify it works

12. Troubleshooting

---

1. What you'll build

A local server that exposes **116 Atlassian tools** (Jira, Bitbucket, Confluence, Bamboo) over the **MCP protocol**, so AI assistants can read/write your Atlassian data through natural language.

Two ways the server can run:

Mode	Transport	Best for
stdio	Direct pipe to a Python process	Claude Code, Cursor
streamable-http	HTTP server on `localhost:8001`	Claude Desktop, ChatGPT, Docker

---

2. Prerequisites — install everything

You need: **Python 3.10+**, **Git**, **Docker Desktop**, **Node.js** (only if you'll use Claude Desktop), and **Claude Code CLI**.

2.1 Install Homebrew (macOS package manager)

Open **Terminal** and paste:

```
/bin/bash -c "$(curl -fsSL
        https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

After it finishes, follow the on-screen instructions to add Homebrew to your `PATH` (it prints two `echo` commands — copy/paste them).

Verify:

```
brew --version
```

2.2 Install Python 3.13

```
brew install python@3.13
```

Verify:

```
python3 --version    # should print Python 3.13.x
```

2.3 Install Git

```
brew install git
git --version
```

2.4 Install Docker Desktop

Option A — via Homebrew (easiest):

```
brew install --cask docker
```

Option B — manual download:

1. Go to https://www.docker.com/products/docker-desktop/
2. Click **Download for Mac** (pick **Apple Silicon** for M1/M2/M3, **Intel** for older Macs).
3. Open the downloaded `.dmg` and drag **Docker** into **Applications**.

After install — start Docker Desktop:

1. Open **Applications** → **Docker** (you'll see the whale icon in the menu bar).
2. Accept the license, sign in (or skip), and wait for the whale to stop animating — that means Docker is running.
3. Verify in Terminal:

```
docker --version
docker compose version
docker run hello-world      # downloads a tiny test image
```

If `docker run hello-world` prints "Hello from Docker!" you're good.

2.5 Install Node.js (only if using Claude Desktop)

Claude Desktop talks to HTTP MCP servers via a bridge tool (`mcp-remote`), which runs on Node.

```
brew install node
node --version    # should be v20+
npx --version
```

2.6 Install Claude Code CLI (optional but useful)

```
npm install -g @anthropic-ai/claude-code
claude --version
```

---

3. Get the project (clone from GitHub)

The project is hosted at **https://github.com/ElafNawaf/atlassian-mcp**.

3.1 Pick a folder for it

```
mkdir -p ~/PycharmProjects
cd ~/PycharmProjects
```

3.2 Clone the repository

Option A — HTTPS (easiest, no SSH setup):

```
git clone https://github.com/ElafNawaf/atlassian-mcp.git
cd atlassian-mcp
```

If the repo is **private**, GitHub will prompt for credentials. Use a **Personal Access Token** as the password — create one at https://github.com/settings/tokens (scope: `repo`).

Option B — SSH (recommended if you'll push changes often):

First, set up an SSH key (only do this once per machine):

```
ssh-keygen -t ed25519 -C "elaf.nawaf7@gmail.com"
# press Enter through all prompts to accept defaults

eval "$(ssh-agent -s)"
ssh-add --apple-use-keychain ~/.ssh/id_ed25519
pbcopy < ~/.ssh/id_ed25519.pub          # copies the public key to clipboard
```

Then add the key to GitHub: https://github.com/settings/ssh/new → paste, save.

Verify and clone:

```
ssh -T git@github.com                    # should say "Hi ElafNawaf! You've
        successfully authenticated"
git clone git@github.com:ElafNawaf/atlassian-mcp.git
cd atlassian-mcp
```

3.3 Confirm the clone

```
ls
# you should see: server.py  tools.py  clients.py  Dockerfile  docker-
        compose.yml  .env.template  README.md  SETUP_GUIDE.md ...
```

3.4 Pulling future updates

When the repo is updated on GitHub, sync your local copy:

```
cd ~/PycharmProjects/atlassian-mcp
git pull
```

If you're running the server in Docker, rebuild after pulling:

```
docker compose up --build -d
```

> **You already have the project** at `/Users/elafnawaf/PycharmProjects/atlassian-mcp`, so you can skip this section. Use it only if you're setting up on a new machine.

---

4. Configure credentials (.env)

The server reads credentials from a `.env` file in the project root.

4.1 Create `.env` from the template

```
cd /Users/elafnawaf/PycharmProjects/atlassian-mcp
cp .env.template .env
```

4.2 Get your Atlassian API tokens

Jira / Confluence (Cloud):

1. Go to https://id.atlassian.com/manage-profile/security/api-tokens
2. Click **Create API token**, give it a label (e.g. "mcp-server"), and **copy the token immediately** (you can't view it again).

Bitbucket Cloud:

1. Go to https://bitbucket.org/account/settings/app-passwords/
2. Click **Create app password**.
3. Tick scopes: `Repositories: Read/Write`, `Pull requests: Read/Write`, `Account: Read`.
4. Copy the password.

Bamboo: Get a personal access token from your Bamboo instance: **Profile** → **Personal access tokens** → **Create token**.

4.3 Edit `.env`

Open it in any editor:

```
open -a TextEdit .env
# or
code .env       # if you have VS Code installed
```

Fill in the values you have. **You only need to fill in the products you actually use** — leave the others as placeholders, the server will simply not call those APIs.

```
WORKGRAPH_MODE=DRY_RUN          # keep DRY_RUN until you trust it; switch to
        EXECUTE later
MOCK_MODE=false

JIRA_BASE_URL=https://yourcompany.atlassian.net
JIRA_EMAIL=elaf.nawaf7@gmail.com
JIRA_TOKEN=<paste-your-jira-token>

BITBUCKET_BASE_URL=https://api.bitbucket.org
BITBUCKET_WORKSPACE=<your-workspace-slug>
BITBUCKET_USERNAME=<your-bitbucket-username>
BITBUCKET_APP_PASSWORD=<paste-your-app-password>

CONFLUENCE_BASE_URL=https://yourcompany.atlassian.net/wiki
CONFLUENCE_EMAIL=elaf.nawaf7@gmail.com
CONFLUENCE_TOKEN=<paste-your-jira-token>     # same token works for
        Confluence Cloud

BAMBOO_BASE_URL=https://your-bamboo.example.com
BAMBOO_USERNAME=<your-bamboo-username>
BAMBOO_TOKEN=<paste-your-bamboo-token>
```

⚠️ **Safety:** While `WORKGRAPH_MODE=DRY_RUN`, write operations only return what *would* happen. Set it to `EXECUTE` only when you're ready for real writes.

5. Choose your run mode

You want to use…	Run mode	Section
Claude Code (CLI)	stdio (Python direct)	§7 + §8
Claude Desktop	HTTP (Docker)	§6 + §9
ChatGPT	HTTP (Docker) + tunnel	§6 + §10

You can run **both** modes side-by-side if you want.

6. Run with Docker (recommended)

Docker keeps the server running in the background, isolated from your system.

6.1 Make sure Docker Desktop is running

The whale icon in the menu bar should be solid (not animating).

6.2 Build and start the server

```
cd /Users/elafnawaf/PycharmProjects/atlassian-mcp
docker compose up --build -d
```

What this does:

- `--build` builds the Docker image from the `Dockerfile`
- `-d` runs it detached (in the background)
- Reads your `.env` and exposes the server on **http://localhost:8001/mcp**

6.3 Confirm it's running

```
docker compose ps              # should show atlassian-mcp running
docker compose logs -f         # tail the logs (Ctrl+C to exit tailing)
curl http://localhost:8001/mcp # should respond (probably with 405 — that's
        fine, means it's alive)
```

6.4 Common Docker commands

```
docker compose stop            # stop the server
docker compose start           # start it back up
docker compose restart         # restart (e.g. after editing .env)
docker compose down            # stop and remove the container
docker compose up --build -d   # rebuild and start (after code changes)
docker compose logs --tail 50  # last 50 log lines
```

---

7. Run locally with Python (no Docker)

Use this for **stdio** mode (Claude Code) or quick testing.

7.1 Create a virtual environment

```
cd /Users/elafnawaf/PycharmProjects/atlassian-mcp
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

7.2 Run in stdio mode (default)

```
python server.py
```

The terminal will appear to hang — that's correct. It's waiting for an MCP client to connect over stdin/stdout. Press `Ctrl+C` to stop.

7.3 Run in HTTP mode

```
MCP_TRANSPORT=streamable-http MCP_PORT=8001 python server.py
```

Now reachable at `http://localhost:8001/mcp`.

---

8. Connect to Claude Code (CLI)

Claude Code launches the MCP server as a subprocess via **stdio** — no Docker needed.

8.1 Edit Claude Code's MCP config

The repo already contains `.mcp.json` at the project root, which Claude Code auto-detects when you run `claude` from inside the folder:

```
{
  "mcpServers": {
    "atlassian": {
      "command": "python",
      "args": ["/Users/elafnawaf/PycharmProjects/atlassian-mcp/server.py"]
    }
  }
}
```

To make it available in **every** project (not just this folder), add it to your global config:

```
claude mcp add atlassian python /Users/elafnawaf/PycharmProjects/atlassian-
        mcp/server.py
```

If you used the virtual env from §7.1, point at the venv's Python:

```
claude mcp add atlassian /Users/elafnawaf/PycharmProjects/atlassian-
        mcp/.venv/bin/python /Users/elafnawaf/PycharmProjects/atlassian-
        mcp/server.py
```

8.2 Test inside Claude Code

```
cd /Users/elafnawaf/PycharmProjects/atlassian-mcp
claude
```

Then in the prompt, type:

> List my Jira projects.

Claude should call `mcp__atlassian__mcp_jira_list_projects`. Use `/mcp` inside the chat to see connection status.

---

9. Connect to Claude Desktop

Claude Desktop only speaks **stdio** to MCP servers. Since our server runs over HTTP (in Docker), we use a small bridge tool called `mcp-remote` that translates between the two.

9.1 Make sure the HTTP server is up

```
docker compose up -d
curl http://localhost:8001/mcp     # should respond (any HTTP code = alive)
```

9.2 Locate Claude Desktop's config file

```
open ~/Library/Application\ Support/Claude/
```

If `claude_desktop_config.json` doesn't exist, create it.

9.3 Add the MCP server entry

Paste this into `claude_desktop_config.json`:

```
{
  "mcpServers": {
    "atlassian": {
      "command": "/Users/elafnawaf/PycharmProjects/atlassian-mcp/mcp-
        bridge.sh"
    }
  }
}
```

The `mcp-bridge.sh` wrapper (already in the repo) ensures Claude Desktop finds a modern Node.js even if your shell defaults to a different version. It runs:

```
npx -y mcp-remote http://localhost:8001/mcp
```

Make sure it's executable:

```
chmod +x /Users/elafnawaf/PycharmProjects/atlassian-mcp/mcp-bridge.sh
```

> **Tip:** If your Node lives elsewhere, edit the path inside `mcp-bridge.sh`. Run `which node` to find it.

9.4 Restart Claude Desktop

Quit completely (⌘Q) and reopen. Look for the 🔌 **plug icon** in the input bar — clicking it should list `atlassian` with all 116 tools available.

---

10. Connect to ChatGPT

ChatGPT supports MCP servers through **Custom Connectors** (in **ChatGPT Pro/Business/Enterprise**, requires Developer Mode for personal accounts). Connectors need a **public HTTPS URL** — your local `localhost:8001` won't work directly. You'll expose it via a tunnel.

10.1 Make sure the Docker HTTP server is running

```
docker compose up -d
```

10.2 Install a tunneling tool — pick one

Option A — ngrok (easiest):

```
brew install ngrok
ngrok config add-authtoken <your-token>     # sign up free at
        https://ngrok.com
ngrok http 8001
```

ngrok prints something like:

```
Forwarding   https://abcd-1234.ngrok-free.app -> http://localhost:8001
```

Your public MCP URL is `https://abcd-1234.ngrok-free.app/mcp`.

Option B — Cloudflare Tunnel (free, no signup needed for quick tunnel):

```
brew install cloudflared
cloudflared tunnel --url http://localhost:8001
```

Copy the `https://*.trycloudflare.com` URL it prints, then append `/mcp`.

10.3 Add the connector in ChatGPT

1. Open https://chatgpt.com → **Settings** → **Connectors**.
2. (Personal accounts) Enable **Developer Mode** under **Settings** → **Connectors** → **Advanced**.
3. Click **Add custom connector** (or **Create**).
4. Fill in:
   - **Name:** `Atlassian MCP`
   - **MCP Server URL:** `https://abcd-1234.ngrok-free.app/mcp` (your tunnel URL + `/mcp`)
   - **Authentication:** None (or add a header if you've configured one)
5. Save. ChatGPT will probe the server and list discovered tools.

10.4 Use it

Start a new chat → click the **+** in the composer → enable **Atlassian MCP**. Ask:

> Show me Jira issues assigned to me.

> ⚠️ **Security:** A public tunnel exposes your local server to anyone with the URL. Use ngrok's basic auth or Cloudflare Access for anything beyond personal testing, and **never share** the tunnel URL.

---

11. Verify it works

Quick sanity tests once everything is connected:

Test	What to ask the AI
Read	"List my Jira projects."
Search	"Search Confluence for pages about onboarding."
Detail	"Get details for issue PROJ-123."
Dry-run write (with `WORKGRAPH_MODE=DRY_RUN`)	"Add a comment 'test' to PROJ-123." → should reply "would add"
Real write (after switching to `EXECUTE`)	Same prompt → comment actually appears

Check the audit log to see every call:

```
tail -f /Users/elafnawaf/PycharmProjects/atlassian-mcp/audit.log.jsonl
```

---

12. Troubleshooting

`docker compose up` fails with "Cannot connect to the Docker daemon" Docker Desktop isn't running. Open it from Applications and wait for the whale to be solid.

`pip install` fails with permission errors You're not in a virtual env. Run `python3 -m venv .venv && source .venv/bin/activate` first.

Claude Desktop doesn't see the server

- Confirm the HTTP server is up: `curl http://localhost:8001/mcp`.

- Check Claude Desktop's logs: `~/Library/Logs/Claude/mcp*.log`.
- Ensure `mcp-bridge.sh` is executable (`chmod +x mcp-bridge.sh`).
- The Node path inside `mcp-bridge.sh` must exist — edit it to match `which node`.

Claude Code says "MCP server failed to start"

- Run `python /Users/elafnawaf/PycharmProjects/atlassian-mcp/server.py` manually — fix any errors it prints.
- If you used a venv, the config must point to `.venv/bin/python`, not bare `python`.

ChatGPT connector says "couldn't connect"

- Make sure the tunnel is still running (ngrok/cloudflared windows must stay open).
- The URL must end in `/mcp`.
- Test the public URL with `curl https://your-tunnel.ngrok-free.app/mcp`.

401/403 from Atlassian APIs

- Re-check the token in `.env` (no quotes, no trailing spaces).
- For Jira/Confluence Cloud, the token pairs with your **email**, not username.
- For Bitbucket Cloud, app passwords pair with your **username**.
- Restart the server after editing `.env`: `docker compose restart`.

Writes don't actually happen You're still in dry-run. Set `WORKGRAPH_MODE=EXECUTE` in `.env` **and** pass `execute=true` in the tool call (the AI does this when you confirm). Restart the server.

View live server logs

```
docker compose logs -f         # Docker mode
# or for local: just look at the terminal running python server.py
```

Quick reference card

```
# Start (Docker)
docker compose up -d

# Stop
docker compose down

# Restart after .env change
docker compose restart

# View logs
docker compose logs -f

# Run locally for Claude Code
python server.py

# Test HTTP endpoint
curl http://localhost:8001/mcp

# Open Claude Desktop config
open ~/Library/Application\ Support/Claude/claude_desktop_config.json

# Tail audit log
tail -f audit.log.jsonl
```

You're done. Ask the AI a Jira question and watch it work.